

Sovereign Agentic OS

The official end-user guide — install, operate, and understand the platform

Orchestrated by Data Masterclass · datamasterclass.com · www.sovereign-agentic.com

Chart version 0.2.7 (app 0.2.0-alpha.8) · generated 2026-06-30 from commit 9a2582f

1 Introduction

- 1.1 What the Sovereign Agentic OS is
- 1.2 Who it is for
- 1.3 The sovereignty / EU-residency thesis
- 1.4 The layered architecture

2 Quickstart

- 2.1 Prerequisites
- 2.2 One command
- 2.3 The two front doors
- 2.4 The demo logins
- 2.5 Smoke test

3 Installation in depth

- 3.1 The wizard (`./install.sh`)
- 3.2 The `mode: bundled | external` contract
- 3.3 The preset files
- 3.4 RAM and the build-and-toggle defaults
- 3.5 Uninstall

4 The front doors

- 4.1 OS UI — the product front door
- 4.2 Admin Console — operate the stack

5 The golden paths

- 5.1 1. Ask an agent (RAG)
- 5.2 2. Query the lakehouse (central Trino / MCP via LiteLLM)
- 5.3 3. Build a dashboard (Superset)
- 5.4 4. Ship software (Forgejo → CI → Argo CD)

6 The golden paths in depth

- 6.1 One model for everything
- 6.2 Data
- 6.3 Files
- 6.4 Metrics
- 6.5 Dashboards
- 6.6 Agents
- 6.7 Science (ML) — opt-in, Layer 4
- 6.8 Software — build governed apps
- 6.9 Connections
- 6.10 Monitoring
- 6.11 Marketplace — internal cross-domain consume
- 6.12 The governance spine
- 6.13 Platform Admin — the tenant control room

7 Component reference

- 7.1 Layer 1 — Agent core
- 7.2 Layer 2 — Context / foundations
- 7.3 Infrastructure backends (shared, mostly Layer 2)
- 7.4 Layer 3 — Self-service & delivery

- 7.5 Security baseline
- 7.6 Platform / UI

8 Security model

- 8.1 Default-deny egress + the proxy chokepoint
- 8.2 OPA tool authorization (default-deny, least privilege)
- 8.3 The governed path to the web
- 8.4 Secrets handling
- 8.5 Secure-by-default pod hardening
- 8.6 Audit trail

9 Deploying to your cloud (STACKIT) {#deploying-to-your-cloud-stackit}

- 9.1 0. Prerequisites in your cloud account
- 9.2 1. Provision the managed resources (Terraform preferred)
- 9.3 2. In-cluster platform (bootstrap, before the OS chart)
- 9.4 3. Configure the OS for managed backends
- 9.5 4. Secrets — never in git
- 9.6 5. Deploy + verify
- 9.7 Cost & scaling

10 Troubleshooting & FAQ

- 10.1 Install & cluster
- 10.2 Operating
- 10.3 Per-component gotchas
- 10.4 Cloud

11 Appendix

- 11.1 A. Full demo-login table (profile `local`)
- 11.2 B. Component / image inventory
- 11.3 C. Pinned versions (agent-core slice)
- 11.4 D. Common ports (local port-forward targets)
- 11.5 E. Version & changelog

1 Introduction

The **Sovereign Agentic OS** is orchestrated by [Data Masterclass](#). Project home and documentation downloads: www.sovereign-agentic.com.

1.1 What the Sovereign Agentic OS is

The **Sovereign Agentic OS** is a self-hostable, EU-residency platform that assembles roughly two dozen best-in-class, permissively-licensed open-source tools into a single governed stack where every business **domain** can create, store, use, document and share data, knowledge, dashboards, agents and software.

It ships as **one umbrella Helm chart** (`charts/sovereign-agentic-os`) that brings up the whole platform on any Kubernetes cluster. The default install is **fully self-contained and works out of the box**: every backend runs inside the chart and a tiny local mock LLM stands in for a model provider, so nothing external — and no API key — is required to see the system working end to end.

The same chart scales from a laptop (a local `kind` cluster) to a sovereign production deployment on **STACKIT** (the EU/Germany cloud), where individual backends are switched to managed services and real secrets arrive through a secrets manager. The only difference between the two is a **values choice** — `mode: bundled | external` — per backend.

1.2 Who it is for

- **Regulated organizations, the public sector, and EU-based enterprises** that need data residency, full audit trails, and zero dependency on US-controlled cloud or SaaS LLM platforms.
- **Teams that cannot send data to hosted AI services** but still want production-grade agentic workflows: RAG agents, a lakehouse, BI, and software delivery — all self-hosted.
- **Data Masterclass participants** building production-grade agentic systems, where the lab environment uses the *same* production components, not teaching forks.

1.3 The sovereignty / EU-residency thesis

Three principles define the platform:

1. **Permissive open source only.** Every bundled component is Apache 2.0 / MIT / BSD / PostgreSQL licensed. Source-available or commercially-restricted tools may be self-hosted by a customer but are *never* bundled into the default distribution. This guarantees full code auditability (critical for sovereign buyers), no proprietary lock-in, and the right to host and modify indefinitely. Concretely, this drove a number of deliberate component choices:

Avoided	Reason	Used instead
Redis	relicensed to SSPL/RSALv2 (2024)	Valkey (BSD-3)
pgvector	one more thing to run; not the retrieval backbone	OpenSearch (vector + lexical in one)
MinIO (shipped)	AGPLv3 copyleft	STACKIT Object Storage (prod) — MinIO is a local-only dev stand-in
Gitea	open-core drift risk	Forgejo (GPLv3+, non-profit Codeberg e.V.)
LangSmith	proprietary; self-host = paid	Langfuse v3 (MIT core)

2. **Security and governance are the operating model, not an afterthought.** The platform ships *secure by default*: agents have no raw internet, every tool call is authorized, every model call is metered and traced, and no real secret ever lives in git.
3. **Self-hosted, in your region.** The production target is STACKIT Kubernetes (SKE) with STACKIT Object Storage in **EU01 / Deutschland Süd**. Model calls can be routed to **STACKIT AI Model Serving** so prompts and completions never leave the sovereign boundary.

1.4 The layered architecture

The platform is organized into layers that you can reason about — and enable — independently.

- **Layer 1 — Agent core.** The agentic runtime: **LangGraph** agents calling **LiteLLM** (the unified model + MCP gateway, with per-key access control and cost caps), every action traced in **Langfuse v3**, retrieving over **OpenSearch** (hybrid vector + lexical — there is no pgvector).
- **Layer 2 — Context / foundations.** Turning raw knowledge and data into governed, discoverable products: **OPA** (policy-as-code at the tool boundary), **Docling** (document parsing), **Haystack** (RAG pipelines), **Dagster** (orchestration), **dbt** (transforms), **Cube** (the metrics/semantic layer), and **OpenMetadata** (catalog + lineage).
- **Layer 3 — Self-service.** Query, explore, visualize, and ship: the **Iceberg** lakehouse (**Polaris** catalog + **central Trino** as the governed query engine for all shared marts — OPA row/column governance + per-domain resource groups; DuckDB kept only for the personal/sandbox lane behind Trino), **Superset** for BI/dashboards, and **Forgejo + Argo CD** for software delivery (git → CI → GitOps; the CI runner builds a container image, pushes it to Forgejo's registry, and Argo redeploys it).
- **Layer 4 — Science / ML (opt-in, off by default).** Traditional ML: **JupyterHub** (notebooks), **MLflow** (experiment tracking + model registry), **Featureform** (feature store, MPL-2.0, optional), and **KServe** (model serving, bootstrap), plus an ML agent. Heavy/GPU-oriented — enable per component as node capacity allows (see *RAM and the build-and-toggle defaults*).
- **Security baseline.** Spanning every layer: **default-deny egress**, a single **egress-proxy** chokepoint, a **governed web_fetch** tool, OPA tool authorization, externalized secrets, and secure-by-default pod hardening.

On top of these sit the **two front doors** you actually open in a browser: the **OS UI** (the product front door for business users) and the **Admin Console** (operate the stack — status, on/off, docs).

Scope note. This release stands up **Layers 1–3** (built incrementally) with **Layer 4 (Science/ML)** available opt-in/off-by-default, under a secure-by-default baseline. The **OS UI is v1.0** — every sidebar tab is a real surface (incl. Science, Metrics, Marketplace, and About/Licenses), styled to the Sovereign Agentic brand with a light/dark theme (light default), and the Admin Console embedded under **Platform** → **Components**. **Strategy** is operational (pillars, RLS-scoped value roll-up, targets, and a live adoption scoreboard); Marketplace and Big Bets are seeded v1 workspaces, and the agent/connector codegen flows are drafts for review. The core is **Apache-2.0 licensed**; bundled components keep their own licenses (see `THIRD-PARTY-LICENSES.md` + `licenses/`). Per-domain spaces and identity (Ory) are the next build.

2 Quickstart

2.1 Prerequisites

- A container runtime + the `docker` CLI (Docker Desktop or Colima), **running**.
- `kind`, `helm`, and `kubectl` on your `PATH`.
- About **14 GB RAM / 6 CPU** available to the runtime VM. The slice is RAM-bound (OpenSearch + ClickHouse + Postgres are the heavy tenants).

2.2 One command

```
# prereqs: docker (running), kind, helm, kubectl
./install.sh           # press Enter through every prompt
```

Pressing **Enter** through every prompt gives the **fully self-contained** install: every backend is bundled (Postgres via CloudNativePG, OpenSearch, ClickHouse, Valkey, MinIO object storage) and a tiny local **mock LLM** answers model calls — nothing external is required. `install.sh` creates the `kind` cluster if needed, bootstraps the operators, builds and loads the bespoke images, installs the chart, seeds the demo data, and finally prints the **demo logins**.

```
./install.sh --defaults      # non-interactive, all bundled (CI / quick)
./install.sh --uninstall     # remove the release (keeps the cluster)
```

(make `install`, make `install-defaults`, and make `uninstall` wrap these.)

2.3 The two front doors

Everything is reachable by port-forward. Start with these two:

```
# OS UI – the product front door (Home / Agents / Knowledge / ... / Consoles)
kubectl -n agentic-os port-forward svc/os-ui 8080:3000      # http://localhost:8080

# Admin Console – operate the stack (status, on/off, addresses, logins + docs)
kubectl -n agentic-os port-forward svc/admin-console 8081:8080 # http://localhost:8081
```

2.4 The demo logins

The **default Administrator-style console is Langfuse**. The full table is in the [Appendix](#); the ones you need first:

Console	Port-forward	URL	Login
Langfuse (traces)	svc/agentic-os- langfuse-web 3000:3000	http://localhost:3000	admin@datamasterclass.com / langfuse-local-dev-admin
OS UI	svc/os-ui 8080:3000	http://localhost:8080	— (no login locally)
Admin Console	svc/admin- console 8081:8080	http://localhost:8081	— (no login locally)
Superset (BI)	svc/agentic-os- superset 8088:8088	http://localhost:8088	admin / superset-admin-local- dev
LiteLLM (gateway)	svc/agentic-os- litellm 4000:4000	http://localhost:4000/ui	admin / litellm-admin-local- dev
Forgejo (git)	svc/forgejo-http 3001:3000	http://localhost:3001	gitea_admin / forgejo-admin- local-dev

These are local dev throwaways (profile `local`), clearly marked as such and never reused on STACKIT, where every secret is external. See [Security model](#).

2.5 Smoke test

```
# Ask the RAG agent a question grounded in OpenSearch-retrieved context, traced in
Langfuse:
kubectl -n agentic-os run ask --rm -i --restart=Never --image=curlimages/curl:8.11.1 --
  curl -sS http://sample-agent:8000/ask -G \
  --data-urlencode "q=What provides the retrieval backbone for vector and lexical
  search?"
```

The response includes the answer, the retrieved knowledge titles, and `traced_in_langfuse: true`.

3 Installation in depth

3.1 The wizard (`./install.sh`)

`install.sh` is a works-out-of-the-box wizard. **Press Enter to accept the default at every prompt** and you get the fully self-contained install (Mode A). The prompts, in order:

1. **Target cluster** — `kind` (default) / `stackit` / `other`. `kind` is the local path; it creates the `agentic-os` cluster if it does not exist and switches your kube-context to it.
2. **All-STACKIT-managed shortcut** — only asked when target is `stackit`. Answering `yes` selects the `values.stackit-managed.yaml` preset (all backends → managed) and tells you to provision the managed services with Terraform first (see [Deploying to your cloud](#)).
3. **Per-backend choice** — for **Postgres**, **OpenSearch**, **Object storage**, and **Cache (Valkey)**, each prompt accepts `bundled` (Enter) or an external endpoint. Anything other than `bundled` is written into the generated overlay as `mode: external` with that endpoint.
4. **LLM endpoint** — `local` tiny mock (Enter) / `openai` / `azure` / `mistral` / `stackit` / `vllm`.
Choosing a real provider also asks for a key/token name, stored as a Kubernetes **secret reference** (never inline).

The wizard then bootstraps the CloudNativePG operator, builds + loads the bespoke images (on `kind`), runs `helm dependency build`, and finally `helm upgrade --install`. The chart's **post-install hooks** seed the demo data automatically (knowledge index, dbt warehouse, Iceberg table, Superset dataset, Forgejo repo → Argo app, Langfuse project + keys). When it finishes it prints the front doors, the demo logins, and a couple of "try it" commands.

3.2 The `mode: bundled | external` contract

Every stateful backend exposes the same toggle. `bundled` (the default) runs it inside the chart; `external` disables the bundled deployment and points the chart at a managed endpoint, with credentials coming from a named Kubernetes Secret (via External Secrets in production). This is **per backend** — you can run managed Postgres but bundled OpenSearch, for example. From `values.example.yaml`:

```
# Omit the block entirely to keep a backend bundled (the default).
objectStorage:
  mode: external
  external:
    endpoint: "https://object.storage.eu01.onstackit.cloud"
    secretName: object-storage-credentials # holds AWS_ACCESS_KEY_ID / SECRET

postgres:
  mode: external
  external:
    host: my-postgres.example.com
    secretName: postgres-credentials

llm:
  mode: external
  provider: azure # openai | azure | mistral | stackit | vllm
  secretRef: litellm-provider-key
```

3.3 The preset files

Preset	Mode	What it does
values.selfcontained.yaml	A (default)	Everything bundled; tuned to fit a single ~14 GB kind VM. This is what ./install.sh uses by default.
values.stackit-managed.yaml	B	Each stateful backend disabled and pointed at a managed STACKIT endpoint; secrets via External Secrets; LiteLLM → STACKIT AI Model Serving.
values.example.yaml	reference	Annotated example of the per-backend mode contract — copy the blocks you need into your own overlay.

The chart's own product defaults live in charts/sovereign-agentic-os/values.yaml (everything enabled, sized for a large STACKIT node). The wizard always writes a small values.generated.yaml overlay and installs `-f <preset> -f values.generated.yaml`, so your choices layer cleanly on top of a preset.

3.4 RAM and the build-and-toggle defaults

The full L1+L2+L3 set is sized for a large (96 GB) STACKIT node. To fit a 14 GB local VM, values.selfcontained.yaml ships a few heavy components **off** by default locally:

```
docling: { enabled: false } # pulls ML models; RAM-heavy
osdashboards: { enabled: false } # ~0.5–1 GB Node.js app
openmetadata: { enabled: false } # heaviest single component (JVM ~2–3 GB)
trino: { enabled: true } # central GOVERNED query engine (off only on a tiny
node)
spark: { enabled: false } # optional distributed batch engine
```

Turn any of them on from the **Admin Console** (runtime on/off, scales 0↔1) or permanently by setting `<component>.enabled: true` and re-running `helm upgrade` (or `./install.sh`). On a large node, the product `values.yaml` keeps everything enabled.

3.5 Uninstall

```
./install.sh --uninstall      # helm uninstall the release; keeps the cluster +  
                             operators  
kind delete cluster --name agentic-os  # remove the whole local cluster
```

4 The front doors

The platform has two browser front doors. Both are reached by port-forward (no login locally).

4.1 OS UI — the product front door

```
kubectl -n agentic-os port-forward svc/os-ui 8080:3000 # http://localhost:8080
```

A Next.js app shell with a left sidebar, **at v1.0: every sidebar tab is a real surface** — no "soon" stubs. Surfaces call the in-cluster backends through **server-side API routes**, so credentials and keys never reach the browser. The tabs and their wiring:

Surface	What it shows / does	Backend
Home	A live stack-status strip (probes ~8 backends) + executable golden-path cards that deep-link to where each path runs (Agents, Data, Dashboards, Software, Science)	/api/status
Strategy	Strategic pillars (shared tenant + optional domain) each tracking a governed business-value metric distributed top-down to its Big Bets → components (RLS-scoped, reconciling), with annual + quarterly targets vs monthly actuals and a live adoption scoreboard (promoted/certified by domain)	/api/strategy/*
Home	The welcoming launcher + cockpit : an illustrated golden-path launcher (one card per path — explainer · role-aware deep-link action · "How it works" tutorial; paths the role can't act on are explained-but-dimmed) surrounded by OPA/RLS-scoped cockpit modules (What needs me · My WIP · Domain pulse · Health & cost · Recent activity · ask-the-assistant), reordered by persona (User/Creator/Builder/Admin). A read + route surface — it reuses Governance/registry/Strategy/Monitoring, never recomputes, and the ask box scaffolds governed drafts (promote/certify stay human, Langfuse-traced). <i>Strategy + Monitoring feeds are mock-stubbed pending those branches.</i>	/api/home/feed , /api/home/ask
Home	A live stack-status strip (probes ~8 backends) + the illustrated golden-path launcher — ten cards (Data · Knowledge · Connections · Agents · Software · Science · Metrics · Dashboards · Big Bets · Marketplace), each with a role-aware launch action and a "How it works" link that opens that path's tutorial	/api/status
Strategy	Strategic pillars + an agentic-transformation readiness heatmap — <i>seeded v1</i>	—
Big Bets	Strategic AI bets (thesis · target value · confidence · backing artifacts) — <i>seeded v1</i>	—
Strategy	Strategic pillars + an agentic-transformation readiness heatmap — <i>seeded v1</i>	—
Big Bets	Initiative roadmaps over real components — a bet is a <i>goal + roadmap</i> with a PM-grade problem statement, linked up to a Strategy pillar + its governed metric and down to components referenced across every tab. A planner proposes a breakdown + dated roadmap and, on approval, scaffolds each via its tab's governed flow (never self-promotes); status auto-derives from each artifact's real lifecycle; a Gantt rolls up on-track/at-risk/blocked; a composition map (lineage + consume-edges) doubles as the value-attribution graph that distributes the pillar metric pillar→bet→component with upstream credit, reconciling back up. OPA-scoped, RLS-scoped, audited. <i>(kind: live OPA/Langfuse; cross-tab sources offline-mock pending the per-tab branches.)</i>	/api/big-bets/* , OPA, Langfuse
Dashboards	Launch into Superset	—
Agents	A three-level agent IDE (Systems → canvas → agent editor): build agent systems three equivalent ways — visual canvas , Monaco text, or an agent-system helper chat, all editing the same Forgejo-versioned <code>system.yaml</code> — then Build (= execute + verify) , run/schedule/toggle,	/api/agents/* , LiteLLM

Surface	What it shows / does	Backend
	fork-to-own, with a per-agent model picker and grants/capability governance. <i>Build/Run execute against the live agent-runtime when a cluster is reachable (every tool call governed via LiteLLM→OPA→Langfuse), falling back to an in-process mock only offline.</i>	
Software	Lists repos + recent CI runs and creates a real Forgejo repo (starter app → push → CI → Argo deploy)	Forgejo API
Science	Layer-4 launchpad — health + links for MLflow / JupyterHub / Featureform / KServe (opt-in)	health probes
Knowledge	The domain's operating manual : general domain knowledge (the pinned base context for every domain agent) + workflow tiles , one per business process. Each workflow is a single versioned <code>workflow.md</code> edited three equivalent ways — an actor-colored swimlane , Monaco markdown , or a derived Mermaid diagram — with per-step actor/I-O/ links (gaps → jump-to-build), decision rules (soft default, mark-as-hard → OPA guardrails), and tacit capture (paste/upload/record-stub → knowledge-agent-compressed). A context layer (unit-chunk → <code>sovereign-embed</code> → OpenSearch; adaptive hybrid knn+BM25 retrieve → rerank by trust/freshness/authority → OPA + DLS scoped → token-budgeted context pack ; Langfuse pinned-vs-retrieved trace) feeds the agents. Creators draft, Builders publish ; publishable to the marketplace; workflow→ agent scaffold handover.	OpenSearch, LiteLLM, OPA, Langfuse
Data	Talk-to-your-data RAG chat (moved here from Agents), SQL query , a catalog , and a per-product dbt agent (draft)	sample-agent, query-tool, OpenMetadata, LiteLLM
Metrics	Cube semantic-layer query (<code>daily_revenue</code>)	Cube
Files	A governed drive : upload/connect (Drive/OneDrive) → folders + tags → auto-index (parse · embed · hybrid index) → search & cite ; promote/certify like Data (OpenSearch DLS); files_retrieve for agents; "Use as" → Knowledge / Data	OpenSearch, LiteLLM, Docling, Dagster, OpenMetadata
Connections	A connections agent drafts connector configs + a connector catalog (build is a <i>draft</i>)	LiteLLM
Marketplace	Internal cross-domain consume surface — discover Admin-certified products of every type; import = a governed grant (read-in-place under your own identity + per-viewer RLS) / fork / template / instance; approval-gated imports route to Governance; lineage-aware deprecate	OpenMetadata, Cube, OPA, OpenSearch, Langfuse, Approvals
Monitoring	Recent agent traces	Langfuse public API
Governance	The control plane — five sections: Approvals inbox (one queue of deploy-review · autonomous-out-of-policy · access/import · egress · promote/certify; Approve executes the governed effect + audit + optional standing policy), Policies (consolidated read-only plane + Admin override), Audit (hash-chained who/when/why + Langfuse), Cost & limits (caps/quotas + over-cap enforcement), Users & access (role-per-domain → OPA; Ory-backed, never raw credentials)	OPA, Langfuse, LiteLLM, Ory

Surface	What it shows / does	Backend
Monitoring	The read/observe plane — five OPA-scoped lenses (runs · pipelines · cost vs caps · system+self-heal · all-tab artifacts), attention-first, with drill-into-trace + run→pipeline→system→artifact correlation	Langfuse, Dagster, LiteLLM, k8s API, KServe/MLflow, OpenMetadata
Governance	The OPA grants matrix (principal × tool, default-deny), each cell re-verified live	OPA
Settings	Deployment identity + enabled components, and Appearance (light/dark toggle, per-device)	—
Components (Platform)	The Admin Console embedded in-app — live status for all ~32 components by layer, on/off toggles, address/login/docs drawer; proxied server-side	admin-console
Gateway (Platform)	Available models + registered MCP tools	LiteLLM
Orchestration (Platform)	Dagster assets + runs	Dagster GraphQL
Consoles (Platform)	Launchpad cards (port-forward command + URL + dev login) for the full tool UIs	—
About / Licenses (Platform)	Bundled open-source components grouped by SPDX license	—

The live data surfaces (status, RAG, query, classify, knowledge-ingest, repo-create, gateway, policy, traces, metrics, Components, **Strategy**) run against the cluster — Strategy's value roll-up reads the **governed Cube metric** and its adoption scoreboard derives **live from the registry + OpenMetadata**; **Marketplace and Big Bets are seeded v1** workspaces, and the agent-builder / dbt-product / connections / software-builder chats produce **draft specs for review**, not live deploys. Every backend URL is an `osUI.*` value, policy, traces, metrics, Components) run against the cluster; **Big Bets** is operational (registry + the six adapters + planner, OPA/Langfuse live, cross-tab artifact sources offline-mock for kind); **Marketplace and Strategy are seeded v1** workspaces, and the agent-builder / dbt-product / connections / software-builder chats produce **draft specs for review**, not live deploys. Every backend URL is an `osUI.*` value, so a surface can be pointed at a different endpoint (e.g. an Ingress host) per environment.

Brand & theme. The UI is styled to www.sovereign-agentic.com: the gold accent `#c8a24a` on a dark `#0c0b0d` palette, the gold **lotus** logo/favicon, and the fonts **Oswald / Marcellus / Rubik** (self-hosted via `next/font`, offline-safe). **Light mode is the default** (white content, black + gold text; the sidebar and top bar stay black in both modes); **dark mode** restores the full brand palette and is opt-in via **Settings** → **Appearance** (the choice persists per device).

Components — the embedded Admin Console. The **Platform** → **Components** tab embeds the Admin Console inside the OS UI. It proxies the in-cluster `admin-console` service **server-side**, so the browser never holds the Kubernetes token, and offers the same capabilities: live status for all ~32 components

grouped by layer, **on/off toggles** (scale 0↔1; "core" items aren't toggleable), and each component's **address + login + docs** in a drawer.

Tutorials — illustrated, hands-on guides per golden path. Every golden path ships **one tutorial**, authored once and reached two ways that resolve the **same** registry entry (they never drift): the Home card's **"How it works"** link and the tab header's **"Tutorial"** link. A tutorial **opens as an overlay in place** (your route + scroll are restored on close) and follows one template: **Hook** → **3–5 illustrated steps** → **"Walk me through it"** → **"You did it"** (with next-path cross-links). "Walk me through it" runs **lightweight coach-marks over the real tab**, one step at a time, locating controls by **stable data-tutorial-anchor ids** (so the highlight can't desync as the UI re-renders). It runs in two modes: **Practice** on the tab's existing personal/sandbox lane — **no governed writes**, nothing persists to real products — then **graduate** to do it **for real**, where OPA/RLS governs every step. Framing is **role-aware** (Creator "create", Builder also "review/promote", User "use"); the core path is identical. Content is **authored in-repo** (`os-ui/lib/tutorials/`) with a cohesive custom illustration set — not auto-generated.

4.2 Admin Console — operate the stack

```
kubectl -n agentic-os port-forward svc/admin-console 8081:8080 #  
http://localhost:8081
```

A single pane over the whole stack, built with the Python standard library (no external deps). It reads live status from the Kubernetes API through a **least-privilege ServiceAccount** and can only do two things: read status, and scale a workload 0↔1. For each component it shows:

- **Status** — `running / off / disabled / starting`, refreshed periodically.
- **On/off toggle** — scales the Deployment/StatefulSet to 0 (off) or 1 (on). "Core" items (Postgres, Argo CD, the console itself) are not toggleable — manage those via chart values.
- **Address, login, summary, and docs** — the 📖 button renders that component's guide in-app; the header links to *Getting started* and *Cloud configuration*.

Off vs disabled. *Off* = installed but scaled to 0 (toggle back on any time). *Disabled* = not deployed at all (set `<component>.enabled: true` + `helm upgrade` to install it). The toggle is a runtime convenience; for a permanent change use chart values.

5 The golden paths

Four end-to-end workflows prove the system works. Each ships seeded so it works immediately after install.

5.1 1. Ask an agent (RAG)

The sample LangGraph agent runs **retrieve (kNN over OpenSearch)** → **generate (via LiteLLM)** → **trace (Langfuse)**. Retrieved context is treated as *data, not instructions* (an injection defense).

```
kubectl -n agentic-os run ask --rm -i --restart=Never --image=curlimages/curl:8.11.1 -- \
  curl -sS http://sample-agent:8000/ask -G \
  --data-urlencode "q=What is the retrieval backbone?"
```

You get an answer, the retrieved knowledge titles, and `traced_in_langfuse: true`. Open **Langfuse** (`admin@datamasterclass.com` / `langfuse-local-dev-admin`) → project *Agent Core* → Tracing → Traces to see the `rag-agent` run with its retrieve + generate spans. (Real prose comes from the self-hosted **Ministral 3** default served by `model-server`; if you disable it the offline **mock model** answers read canned — swap in any model in LiteLLM with no agent change.) You can also do this in the OS UI **Agents** tab. Edit the seed knowledge via `sampleAgent.knowledge` in values — it is re-ingested on restart.

5.2 2. Query the lakehouse (central Trino / MCP via LiteLLM)

The **query-tool** runs read-only SQL **through central Trino** over the Iceberg marts and is registered in the LiteLLM **MCP gateway** as the OPA-gated `query` tool. OPA gates tool access; **Trino enforces row/column governance** (row filter by domain, column mask by sensitivity) on every read, so agents query data through the same governed endpoint as the BI layer.

```
# Direct HTTP against the governed query tool (dbt-trino mart:
  analytics.daily_revenue):
kubectl -n agentic-os run q --rm -i --restart=Never --image=curlimages/curl:8.11.1 -- \
  curl -sS http://query-tool:8000/query -H "Content-Type: application/json" \
  -d '{"sql":"select order_date, revenue from daily_revenue order by 1"}'
```

Agents reach the same tool via LiteLLM's MCP endpoint (`http://agentic-os-litellm:4000/mcp` , tool `sovereign_query-query`), and OPA decides whether the calling key is allowed. In the OS UI, the **Data** tab is the table browser + SQL surface (governed, via Trino). The **My data** lane gives a private DuckDB sandbox — uploads + a masked pull-extract through Trino — never the governed marts.

5.3 3. Build a dashboard (Superset)

```
kubectl -n agentic-os port-forward svc/agentic-os-superset 8088:8088 #  
http://localhost:8088
```

Log in as `admin` / `superset-admin-local-dev`. A database connection (`warehouse`) and a dataset on `analytics.daily_revenue` are seeded. **Datasets** → **daily_revenue** → **Explore**, build a chart (e.g. revenue by day), and save it to a dashboard. The data is produced by **dbt** (`raw_orders` → `stg_orders` → `daily_revenue`) and the same metrics are defined once in **Cube**, so dashboards and agents share one definition of `revenue / orders`.

5.4 4. Ship software (Forgejo → CI → Argo CD)

```
kubectl -n agentic-os port-forward svc/forgejo-http 3001:3000 # http://localhost:3001
```

Log in as `gitea_admin` / `forgejo-admin-local-dev`. A `demo-app` repo with a CI workflow is seeded. In the OS UI, the **Software** → **app** → **Code** panel gives an **in-browser editor** (Monaco, self-hosted — no CDN) over the app's repo: browse the file tree, edit, and **Save** commits straight back to Forgejo on `main` (Builder/Admin-gated; CI → Harbor → Argo CD then pick it up). A **git push** triggers **Forgejo Actions**, which the **CI runner** executes (Docker-in-Docker): it builds a container image, pushes it to Forgejo's built-in OCI registry, and commits a manifest bump (marked `[skip ci]` so the bump does not re-trigger CI). **Argo CD** sees the change and redeploys the new image into the `demo` namespace.

```
# Watch CI tasks:  
curl -u gitea_admin:forgejo-admin-local-dev \  
http://localhost:3001/api/v1/repos/gitea_admin/demo-app/actions/tasks  
  
# Argo CD (GitOps):  
kubectl -n agentic-os port-forward svc/argocd-server 8082:80 # http://localhost:8082  
# password: kubectl -n agentic-os get secret argocd-initial-admin-secret \  
# -o jsonpath='{.data.password}' | base64 -d
```

The OS UI **Software** tab summarizes repos and recent CI runs. On first install the `demo-app` pod may show `ImagePullBackOff` until the first CI run builds and bumps the image tag — that is expected. In production, the privileged DinD builder is swapped for rootless kaniko/buildah pushing to **Harbor** (scan + sign).

6 The golden paths in depth

The walkthroughs above show the seeded demos. This chapter describes the **operating model** — how you actually load data, build agents, do ML, build software, and connect to external systems — and how each is governed.

Scope: this is the end-to-end model the OS is built around. Layers 1–3 are in place; **Science (Layer 4) is opt-in and off by default**; some surfaces (per-domain spaces, identity, and parts of the Governance approval UI) are on the near-term roadmap — see *Version & changelog*. The **internal cross-domain Marketplace** (consume surface) is now built — see *Marketplace* below.

6.1 One model for everything

Every capability is an **artifact** with the same attributes — **owner · domain · type · visibility**. Whatever the type (data product, knowledge, file, agent, software, ML model, feature set, connection, dashboard), the lifecycle is the same: **Create** → **Document** → **Use** → **Promote**, through the OS UI, scaffolding the real tools underneath, preview-first, cataloged and audited.

Promotion ladder (role-gated):

Visibility	Meaning	Who can promote
Personal / private	the creator only (default for drafts + app-created data/files)	—
Domain (Shared)	usable across the owning domain	Builder or Administrator
Marketplace (cross-domain)	discoverable by other domains	Administrator only

Roles: **User** consumes · **Creator** builds (drafts) · **Builder** certifies + shares in-domain + approves go-live/connection-writes · **Administrator** sets tenant guardrails + promotes to the Marketplace.

6.2 Data

The **Data** tab opens on **tiles, one per dataset** — name, owner, freshness, a quality badge and **Bronze / Silver / Gold** dots. Double-click a tile and the dataset opens as a **three-step pipeline**: it is one logical dataset with three versions of itself.

- **Bronze** — "Bring it in." Upload a file, or pull a slice of a governed product (it arrives already masked to what you're allowed to see). Preview the rows, then confirm.
- **Silver** — "Clean it up + set the key." Fix types, remove duplicates, set the ID.
- **Gold** — "Make it ready." Harmonize to the shared shape and add data-quality monitoring.

Each step is **guided and previewed before it commits**. If a layer isn't needed you can **pass through** it. A **"Show the code"** toggle reveals the real artifact (a dlt pipeline, a dbt model, a Cube model), and the **data agent** can do any step from chat — three ways to shape one file.

Sharing is **role-gated**, like a review/merge. You start with a private **Dataset** (in your own fast DuckDB workbench). Share it and a **Builder promotes** it to a governed **Data asset** in Trino; an **Administrator certifies** it to a **Data product** listed in the **Marketplace**, where other domains import it. Nothing enters the governed store without **documentation + passing tests** — the **transparency gate** turns green only when every artifact is documented and in the lineage graph.

Define a metric on Gold (you only **name the measure**; the rest is scaffolded), build a **dashboard** on it, and the **data agent** answers questions over your domain's data, the products you've imported, and your own private datasets — never anyone else's. Because the agent reads the **same** governed tables and metrics as the dashboards, the numbers never diverge. Two tiers stay separate underneath: **Supabase** holds operational/app state; **Iceberg** on object storage holds the analytical products, with **central Trino** the one governed engine over the lake. (*Engine-room detail: `os-ui/lib/data` .*)

6.3 Files

The **Files** tab is a **calm governed drive** for any **unstructured** file — documents, images, video, audio, archives. You **upload** (drag-drop, any type) or **connect a source** (Google Drive / OneDrive, adding a folder or the whole drive), **organise** with folders + tags, **preview** in place, and **search** across names, tags and content. Everything else is hidden: each file is **auto-indexed** (parsed by type — Docling for docs, transcription for audio/video, OCR/caption for images — then embedded and put in a hybrid **OpenSearch** index) so agents can **retrieve and cite** it; a status chip just shows *Processing* → **Searchable** ✓. Files are **governed exactly like Data** — the same tiers (**private dataset** → **domain asset** → **marketplace product**), the same roles (**Creator** uploads, a **Builder** promotes, an **Admin** certifies) and the same **policy compiler**, here compiled to **OpenSearch document-level security**: a non-member is denied by the *filter*, not by UI logic. Sensitive or huge files can be **stored-but-not-indexed** (and `restricted` files always are). Connected drives **re-govern** under our tiers (not the source's ACLs), sync **copy-into-store or index-in-place** per source, and refresh **overnight then incrementally** (by content hash, so nothing is re-embedded needlessly). Agents reach files through the governed `files_retrieve` tool — hybrid retrieval, DLS- scoped to the **delegated user**, reranked by **trust + freshness + authority**, returning cited text/transcript/caption passages (the original opens on demand). Finally, **"Use as"** distils a file into **Knowledge** (a tacit note) or **Data** (a guided Bronze dataset import), with lineage recorded in OpenMetadata. (*In this pre-release every backend runs **live when the service is reachable** and falls back to a deterministic in-process mock for local `kind` , behind one interface.*) In **Data** → **New data product** you **load** (file/connection/Supabase snapshot), **transform** (dbt models + tests), **document** (cataloged in OpenMetadata with lineage), define **metrics** (Cube), and build **dashboards** (Superset). Two tiers stay separate: **Supabase** holds operational/app state; **Iceberg** on object storage holds the analytical data products. **Central Trino** is the governed engine over the lake (dbt-trino builds the marts; Cube, Superset and the agent `query` tool all read through it). Agents read the *same* marts + metrics as the dashboards, so the numbers never diverge. Defining a metric on a Gold

product **hands off** into the **Metrics** tab (below), where the full "define · explore · govern" experience lives.

6.4 Metrics

The **Metrics** tab is the **KPI semantic layer**. On a Gold data product's auto-cube (the Data tab scaffolds the cube's dimensions from the dbt manifest via `cube_dbt`), you **define a measure** — "Revenue = sum(net_amount)" — three equivalent ways that produce the **same artifact**: a friendly **form**, the **metrics agent** ("define revenue on sales"), or hand-edited **Cube YAML**. Each metric has one **canonical member** (e.g. `Sales.revenue`) that the **explorer**, **Dashboards** and the agent `metrics` tool all resolve — so the numbers are the same everywhere, by construction. The **metric explorer** slices a metric by dimensions with **no SQL** and **per-viewer row-level security** (Cube's security context — two viewers see different rows); analysts can **drop to SQL/Trino**. Metrics are governed products on the **same tiers** as data — **Personal** → **Domain (Builder)** → **Marketplace (Admin)** — and promotion runs a **consistency check** (documented + resolves on its member). Lineage (mart → measure → view) reaches OpenMetadata via a dbt **exposure**. *(Build executes against live Cube when the cluster is reachable, else an honest offline-mock — labelled either way; a ✓ always means a real apply + verify.)*

6.5 Dashboards

The **Dashboards** tab is **governed BI on metrics**. Dashboards are **tiles** (Mine / Domain / Marketplace); **double-click** opens one inline via the **Superset Embedded SDK** and a server-minted **guest token that carries the viewer's RLS in the token** — so a shared dashboard still shows only the viewer's rows. You build a dashboard **two ways at once (dual-mode)**: **drag-and-drop** charts in Superset, or ask the **dashboard agent** ("build me a Sales overview", which drives Superset's REST/MCP) — both edit the **same** dashboard, on **governed metrics** from the Metrics tab (ad-hoc can hit Trino). Set **alerts** (a threshold on a metric → **notify** by email/Slack/in-app **and** trigger a governed **agent run**, traced in Langfuse) and **scheduled reports** (a dashboard snapshot on a cadence). Same governance as everything else — **Personal** → **Domain** → **Marketplace**, per-viewer RLS, OM lineage (metric → chart → dashboard) — and because charts resolve the **same Cube metric** the agents use, **the BI layer and the agents never disagree**.

6.6 Agents

The **Agents** tab is a **three-level IDE** — a list of **agent systems**, a per-system **canvas** (supervisor + members with derived routes), and a focused **editor** for each agent — with **three equivalent editing modes** (drag/connect SVG canvas, self-hosted Monaco text, and an agent-system helper chat) that all edit the *same* Forgejo-versioned `system.yaml`. **Build = execute + verify**: it runs the compiled system and checks it, with **every** model/connection/tool call routed through the **governed gateway** (no agent reaches a capability it was not granted). You pick a LiteLLM model per agent, manage grants/routing, **run / schedule / toggle** the system, and **fork-to-own**; a **validation gate** must pass before a system can build or run. *(Build/Run execute against the live agent-runtime when a cluster is reachable — every*

*model/connection/tool call governed via LiteLLM→OPA→Langfuse — and fall back to an **in-process mock only offline**, when no cluster is present.)*

When you define an agent you write behaviour (`AGENT.md`) and memory (`MEMORY.md`), then **grant the resources** the agent may use — data products, knowledge, files, connections — and the tools it may call. **The one rule:** an agent never touches a raw resource; every call goes through the **model gateway + policy engine (OPA)**, is **cost-capped** and **traced**. Short-term memory is per- conversation; long-term memory is a governed, domain-scoped artifact. By default agents are **read-only / propose-don't-commit** — publishing data/knowledge, writing to external systems, sharing, overspend, or deletes require **approval** by a Builder/Admin. Inside a domain, agents collaborate as a **LangGraph** team; across domains they call each other as **governed tools**.

6.7 Science (ML) — opt-in, Layer 4

In the **Science** tab (off by default, `ml.enabled` per domain) you take **traditional ML** end to end, **preview-first** (the guided **New model** flow + an ML agent drive the no-code common path; JupyterHub notebooks are the escape hatch for power users). Explore a data product, build reusable **features** (Featureform — offline on Iceberg, online in Valkey), **train + track** experiments (MLflow, repeatable as a **Dagster** job), register and compare **models**, and — after a Builder **certifies + approves go-live** — **deploy** to **KServe**. This is classic ML (regression, classification, forecasting, clustering), **not LLMs**.

Model-as-service, tier-gated. A deployed model is exposed **two ways from one KServe endpoint** — a governed **REST predict API** (Software apps / external) **and** a governed **predict MCP tool** (agents) — both through **OPA + LiteLLM**, capped and **Langfuse-traced**. There is **no separate publish step**: who may call follows the **same visibility ladder** as every artifact — **Personal** (owner only) → **Domain** (a Builder promotes; the whole domain may call) → **Marketplace** (an Admin certifies; cross-domain). Promoting/certifying **automatically widens callable scope** via the policy compiler → OPA, so the REST and MCP doors can never diverge. **Certify + go-live are always a Builder/ Admin — never the agent:** the **ML agent** is guided AutoML (propose → features → train → deploy to **Staging**) under two modes (in-tab inline approval / autonomous safety presets + GPU quota), but it never self-certifies or self-promotes. At certify the owner sets the Marketplace **consumption mode** — **read-in-place** (call the shared `predict` without copying, default) or **fork-allowed** (fork-to-retrain). Science also keeps its own **model-monitoring** view (metric history, drift charts, version compare, **retrain trigger**) reading the **same** MLflow/KServe/Dagster signals as Monitoring. GPU is optional, CPU-default and cost-gated.

6.8 Software — build governed apps

In the **Software** tab, **New software** opens a **chat dedicated to that one app**. You build it in plain language with the in-app **OpenCode** assistant beside a self-hosted **Monaco** editor over the app's own Forgejo repo (one of four templates: web app — Next.js + Supabase — service, script, or dashboard). All of the app's **design decisions, data descriptions and documentation live under that app**, versioned in git; data and files it creates are **Personal** to you by default.

You iterate in a **private preview** — a sandbox you run yourself, no review. Going **live on a domain subdomain is Builder-reviewed**: requesting a deploy opens a **review card** carrying the **security scan** (SAST · dependencies · secret-scan), the **governed resources** the app declares, its **cost/resource footprint** (~\$/mo), and the **change diff**. Only a **Builder/Admin in the app's domain** can approve (a creator cannot self-approve), and a **leaked secret or high/critical finding blocks the go-live**. Once approved, the exact scope is recorded as the app's **envelope**: routine in-envelope updates **auto-deploy**, while anything that **broadens scope** (a new write tool, connection, data or knowledge grant, or a higher footprint) re-opens the review.

Building the app **auto-creates its MCP** from the app's **OpenAPI** spec: every read operation is enabled, every **write is held for approval (reads-on / writes-off)**, and that capability profile is compiled into the **same OPA Connection gate** every other connection uses — so an app tool is governed identically (reads allow, writes require approval, anything undeclared is denied) and is instantly usable as a **tool by your agents**. The metadata convention (`app.yaml` , `/.app/` docs, the OpenAPI spec) is **re-parsed on every commit**, so whatever is committed shows up in the app; an imported or legacy repo is still wrapped as a governed app, deriving what it can and prompting for the rest.

Apps **consume governed resources** — a granted Connection, Data product, Knowledge, or another app's MCP — always as a **reference, never raw credentials** (external creds stay in the Connection; app secrets in External Secrets). **Use as Data** snapshots the app's operational data into a Bronze dataset for the Data golden path. Lifecycle is governed too: **archive** disables the app + its MCP but **retains** its data (restorable); **delete** is **lineage-aware** and is blocked while another app still depends on it. Promote it to Shared (Builder/Admin) or the Marketplace (Admin) as usual.

There are **four authoring front doors** — the in-app build chat, the **Platform MCP**, a direct **git push**, and a **git import** of an external repo — and they all converge on the *same* governed commit → metadata-parse → auto-MCP → review pipeline. The **Platform MCP** is the governance invariant this build rests on: it gives **full capability parity with the UI**, governed **identically** under the caller's **delegated identity** — a **front door, never a back door**. It cannot bypass roles, the deploy review, OPA, secrets/egress, or the audit trail, because every MCP tool delegates to the exact same governed function the UI calls.

6.9 Connections

A **Connection** is a governed bridge to an outside system — `credentials + endpoint + a set of governed tools` , never a raw pipe. The **secret lives only in the secrets store**; the agent or app never sees the token — you grant **use**, never the credential. **One connection is used two ways**: as an **agent tool** and as a **data source** (a database both answers a `query` tool **and** lands in the Bronze lake; a drive both serves files **and** indexes into Files).

Who connects. Any user can connect their **own personal account** — e.g. *my* Google Drive or OneDrive — via per-user **OAuth**; it stays private to them. **Builders/Administrators** create **shared** connections with service credentials. The ladder is **Personal** → **Shared (Builder/Admin)** → **Marketplace (Admin)**; the Marketplace shares the **template** (consumers bring their own credentials), so no secret leaves the owner.

The launch set — Google Drive / OneDrive, PostgreSQL / MySQL, any REST API (import an **OpenAPI** spec → governed tools), an external **MCP server**, and SaaS (Notion, Slack) — all sit behind one **adapter interface** (`connect · test · tool-generation · capability-OPA · sync`), so a new integration is just a new adapter. Each works **live when deployed** and falls back to a labelled mock in a local cluster.

Capability profile per tool. Pick a mode — **Off / Read / Write-approval / Write-bounded / Blocked** — plus scope, rate, cost and argument limits. **Reads on, writes off by default.** The profile **compiles to an OPA policy**; agents see only the enabled, in-scope tools, and a grant to an agent can only **restrict**, never broaden.

Write-back has two modes. With a **human present** (in-tab assistants), a write pauses **inline** with a full preview — the action, the **before/after diff**, who asked and why — for the owner or a Builder to approve; **"approve & remember"** turns it into a bounded standing policy so identical calls stop prompting. For **autonomous agents** (no human at run time), a Builder pre-sets a **safety preset (Read-only → Read+propose → Read+bounded-writes → Full-in-scope)**; anything out of policy is **blocked, logged and queued** for review in the Governance tab — never an inline prompt.

Egress is default-deny. Connections may only reach **Admin-approved** endpoints; a Builder **requests** a new endpoint and an Admin **approves** it, and all outbound traffic is logged. Per-connection **health** is shown, with silent OAuth refresh and a **Reconnect** prompt on hard failure.

6.10 Monitoring

The **Monitoring** tab is the **read/observe plane**: one place to **see health, watch spend, and trace any run** across the platform. It is **read-only** — it shows what's happening and surfaces operational alerts; it does **not** set policy or caps (that's **Governance**) and does **not** watch business KPIs (that's **Dashboards**). The clean split is *observe (Monitoring) · decide (Governance) · the business (Dashboards)*.

Five lenses, each scoped by the viewer's **Ory→OPA** identity: (1) **agent & run observability** (Langfuse), (2) **data-pipeline health** (Dagster runs + dbt tests/freshness), (3) **cost & usage** (LiteLLM spend by model/agent/domain **vs the Governance caps** — Monitoring *watches* the spend, Governance *sets* the cap), (4) **system & cluster health + self-heal** (k8s workload status read live via the OS-UI ServiceAccount; Prometheus/Alertmanager/Argo/SKE auto-repair surfaced), and (5) **artifacts across all tabs incl. ML** (MLflow/KServe serving, drift, latency, tied to OpenMetadata lineage). The overview is **attention-first** — the few red/amber items lead, not a wall of green.

Drill-into-trace is the core promise: any run/agent/pipeline/model opens its full Langfuse trace (steps · tool calls · the context pack · inputs/outputs) + **logs**. A **correlation** spine ties a signal back through **run → pipeline → system → artifact** and cross-links to its **Governance** audit entry and the **cost cap** it spends against. **Scope:** a **User** sees only their own runs/cost/ artifacts, a **Builder** their domain, an **Admin** the tenant + cluster — and a User **cannot** open another user's trace (enforced server-side before any step or log is returned). Operational alerts are **system/run health only** (self-heal-or-notify); business-KPI alerts live in Dashboards. Each adapter reads **live** where the backend is up and falls back to an honest **offline mock** otherwise (no AGPL Grafana in the bundle; STACKIT Observability is the Mode-B overlay). See `os-ui/lib/monitoring/README.md`.

6.11 Marketplace — internal cross-domain consume

The **Marketplace** is the *Consume* counterpart to every Build tab's **certify** step: one place to discover and reuse **Admin-certified products of every type** across the tenant's domains (data products · metrics · knowledge · files · apps · dashboards · connections · agents). It is the **internal** tenant marketplace — distinct from listing the OS on STACKIT's external marketplace.

Discovery. Anyone may **browse**: search and filter by **type · domain · tag**. Each listing shows the **certification badge · owner · domain · description · lineage · quality/freshness · usage & ratings · a preview/sample** — so a consumer can judge trust before importing.

Import = a governed grant. The unifying idea: the owner's certified artifact stays the **source of truth** and is consumed **under the consumer's identity + row-level security**. Per type:

Product	Import
data · metric · knowledge · files · dashboard	read-in-place via a policy-compiler grant (Trino+OPA / Cube RLS / OpenSearch DLS), per-viewer RLS — two domains see different rows — plus optional fork-to-adapt
app	deploy your own instance
connection	template — bring your own credentials
agent	fork-to-own — an editable copy

Governance. Open products **auto-grant** (RLS still scopes the rows); imports that touch the owner's live creds/compute (a connection template, a shared app instance) — or that the owner marks approval-required — surface as **import requests in the Governance inbox**, routed to the owner's domain. Clearing the request activates the grant. The **owner sees usage** (who imported, in which mode); everything is **audited** (OpenSearch + Langfuse).

Lifecycle. An **Admin certifies** a product in its own tab → it's **listed** here. **Deprecate/decertify is lineage-aware**: importers are **warned** and the listing is flagged, but an in-use product's grants are **kept** — it can't be silently removed.

Under the hood. OpenMetadata /data-marketplace (data/metrics discovery + lineage) + the OS artifact registry (everything else) + the policy compiler (OPA marketplace package) + Cube/DLS + Langfuse + the Approvals queue. The adapters run **live-or-offline-mock** (authoritative with no cluster), so the whole flow works on local `kind`. Module + API reference: `os-ui/lib/marketplace/README.md`.

6.12 The governance spine

One **gateway** (every tool/model call, cost caps), one **policy engine** (OPA — `allow` / `deny` / `requires_approval`), one **trace** (Langfuse), one **audit**. Two layers of policy: **tenant guardrails** set by Administrators (default-deny egress, no plaintext secrets to agents, no cross-domain data without a grant, model allowlist) that domains cannot override, and **domain policy** set by Builders within those guardrails. High-stakes actions queue for approval in the **Governance** tab.

The **Governance control plane** consolidates this: every side-effectful, role-gated action across the platform (a Software deploy-review, an autonomous out-of-policy action, an access/import request, a new egress endpoint, a promote/certify) surfaces as a **card** in one async **Approvals inbox**. The key principle — **an approval is an action**: on Approve the platform **executes the governed effect** (Argo deploy · policy-compiler grant · egress allowlist · promote/certify · run the queued action) **and** writes audit + (optionally) a **standing policy** ("approve & remember"). Scope follows role: **Users** see their own requests, **Builders** their domain's queues/policy/audit/memberships, **Admins** the whole tenant. Roles are **User · Creator · Builder · Admin**, assigned **per domain** and **compiled to OPA** so a role change takes effect everywhere; identities are backed by **Ory** (the tab never handles raw credentials). Inline (attended) write-approvals stay in the run; only the **async** items queue here. See `os-ui/lib/governance/README.md`.

6.13 Platform Admin — the tenant control room

A **tenant-scoped, Admin-only** area at `/platform`, **above** the per-domain workspace. It is where the tenant's **structure and platform operation** are authored — and it stops short of anything per-domain, which stays in Governance/Monitoring. The boundary is deliberate: **Platform Admin configures, Governance enforces & sees, Monitoring watches — cross-link, never duplicate**.

Ten sections: **Overview** (admin cockpit — component health, spend vs envelope, users/domains, open alerts), **Domains** (create / rename / archive / transfer; per-domain optional-layer toggles `ml.enabled` / `spark.enabled`; templates), **Users & Access** (org-wide invite / deactivate via Ory — no password is ever seen; tenant Admin + initial memberships; in-domain role changes stay in Governance), **Cost & Billing** (the tenant envelope + STACKIT premium cap; operational sub-caps stay in Governance, live spend in Monitoring), **Models & Providers** (the LiteLLM catalog — defaults per task, per-model enable/caps; provider keys **via the secrets manager, never raw**), **Components & System** (the Admin Console — versions, enable/disable, self-heal status, node/pool view), **Security & Egress** (the Admin-curated egress allowlist, secrets-manager status, data residency, OPA policy-bundle version, audit retention), **Backups & Restore** (schedules, last status, an admin-triggered **guarded** restore), **Plugins** (curate/install MCPs/skills/tools + external STACKIT marketplace registration), and **Settings** (SSO/identity via Ory, branding, defaults, localization EN→DE).

The coupling that makes it sovereign-safe: identity, models and egress set here **compile through the policy compiler** → **OPA**, the same default-deny decision engine every tab enforces — so configuring a right here is not a governance bypass. Four invariants hold throughout: **never surface a raw secret** (Ory + the secrets manager; keys stored as a reference + fingerprint), **guarded + confirmed + audited** restore and destructive toggles, **no prod provisioning from the UI** (toggles only scale already-provisioned workloads), and **multi-tenant isolation** (one tenant's admin never sees another). Every action lands in the shared **Audit** record Governance surfaces.

7 Component reference

A section per component, grouped by layer. Each entry summarizes what the component is, how to reach it, how to log in, and the key tasks — see `docs/components/<id>.md` for the full per-component guide. Unless noted, all access is `kubectl -n agentic-os port-forward ...`.

7.1 Layer 1 — Agent core

7.1.1 LiteLLM — model & MCP gateway (`litellm`)

The one governed endpoint agents call for **both** models and MCP tools: per-key access + cost caps, every call logged to Langfuse, MCP tool servers fronted here. DB-backed (CNPG `litellm`).

- **Access:** `svc/agentic-os-litellm 4000:4000` → admin UI `http://localhost:4000/ui`, API docs `/docs`.
- **Login:** `admin` / `litellm-admin-local-dev`; master key `sk-litellm-local-dev-master`.
- **Key tasks:** call models (`sovereign-default` → Ministral 3, `sovereign-reasoning` → in-box Magistral 24B, `sovereign-reasoning-fast` → STACKIT Qwen, `sovereign-embed`, `sovereign-vision` / `sovereign-premium` → STACKIT; `sovereign-mock` is a back-compat alias for the default); manage virtual keys + cost caps; register MCP tool servers. Agents use the scoped key `sk-agents-local-dev` (alias `sovereign-agents`).

7.1.2 Model serving — self-hosted LLMs (`model-server` + `magistral-reasoning`)

The default chat backend is a self-hosted, OpenAI-compatible **Ollama** runtime (`model-server`) serving **Ministral 3 3B** (`ministral-3:3b-instruct-2512-q4_K_M`) — the light tier for chat, coding, and tool-selection — fully offline, no provider key, with `modelServer.replicas` pods behind LiteLLM load-balancing.

The **reasoning** tier is a second self-hosted runtime (`magistral-reasoning` , `modelServer.reasoning`): **Magistral Small 24B** (`Magistral-Small-2506-Q4_K_M.gguf`) on a **llama.cpp** `llama-server` , registered with LiteLLM as `sovereign-reasoning` — the **sovereign** default for planning/deep reasoning, zero per-token, in-box. It is a single **resident** replica and **core-capped**: `modelServer.reasoning.cores` is *both* the CFS CPU limit *and* the llama.cpp `-t` thread count, so it can never starve co-located workloads (e.g. Trino) on a shared box. It is intentionally **slow on CPU** — LiteLLM gives it a generous ~300 s timeout but still trips the breaker on a connection failure and falls over to `sovereign-reasoning-fast` (STACKIT Qwen — the explicit *fast* alternative, pay-per-token, spend-capped). The GGUF is **provided on the box** and mounted at runtime (not redistributed); per-agent the **Reasoning target** picker switches Local-sovereign / Fast / Light.

LiteLLM routes the full fallback chain (self-hosted → STACKIT last-resort / vision) with retries, circuit-breaking, and a per-model spend cap. Swap the default with `modelServer.model` , or disable a tier (`modelServer.enabled` / `modelServer.reasoning.enabled: false`).

License note: Ministral 3 and Magistral Small 24B ship under **Apache-2.0** (OSI-permissive), so the self-hosted tiers are **Apache-clean**. We ship only the Ollama (MIT) and llama.cpp (MIT) engines; the weights are pulled/mounted at runtime and **not redistributed** (see `THIRD-PARTY-LICENSES.md`). To swap models, keep to permissively-licensed weights and size the resources to the tag.

7.1.3 Mock model — offline embeddings + fallback LLM (`mock-model`)

A tiny, dependency-free OpenAI-compatible server (chat + deterministic-hash embeddings). It now backs the **offline embeddings** route (`sovereign-embed`) and serves as the zero-dependency chat fallback when `model-server` is disabled. Reached only through LiteLLM (`http://mock-model:8080/v1`).

7.1.4 Query tool — central Trino engine (MCP) (`query-tool`)

Runs read-only SQL through central Trino over the Iceberg marts; registered in the LiteLLM MCP gateway as the OPA-gated `query` tool (Trino enforces row/column governance). See [golden path 2](#). Locally Trino uses static S3 creds (the local S3 stand-in lacks AWS STS for Polaris credential vending); on STACKIT, vended credentials are enabled. DuckDB is scoped to the personal/sandbox lane (`sandbox-duckdb`), never the governed marts.

7.1.5 Sample RAG agent (`sample-agent`)

The LangGraph agent that proves the core loop (retrieve → generate → trace). Seed knowledge describes the OS itself. `GET /ask?q=...`. Edit `sampleAgent.knowledge` in values to change the corpus.

7.1.6 Poet agent (`poet-agent`)

A second LangGraph agent (compose → save) that writes a poem to a PVC each run — an "open a file to see it worked" demo, same architecture as the RAG agent. `GET /write?topic=...`; pull results with `./scripts/get-poems.sh` (copies to `./poems/`).

7.2 Layer 2 — Context / foundations

7.2.1 OPA — tool authorization (`opa`)

Open Policy Agent makes **default-deny** authorization decisions at the tool boundary: a principal may invoke a tool only if granted. Internet tools (`web_fetch`) are intentionally ungranted by default.

- **Access:** `svc/opa 8181:8181`; query `POST /v1/data/agentik/authz/allow`.
- **Grant a tool:** add it under the principal in `opa.grants`, then `helm upgrade`.

7.2.2 Docling — document parsing (`docling`)

`docling-serve` converts uploaded documents (PDF/DOCX/HTML...) into clean markdown for the knowledge index. **Off by default locally** (RAM); on for STACKIT. `svc/docling 5001:5001`, `POST /v1/convert/source`.

7.2.3 Haystack — RAG retrieval pipeline (`haystack`)

Runs the RAG retrieval pipeline over OpenSearch, embedding via LiteLLM (`sovereign-embed`); uses its own `haystack_knowledge` index. A reusable retrieval service agents can call. `GET /retrieve?q=...`.

7.2.4 Dagster — orchestrator (`dagster`)

Orchestrates the data tier: loads the dbt project as assets and runs `dbt build`. arm64-native image, backed by CNPG `dagster`. **Access:** `svc/agentic-os-dagster-webserver 3070:80` (no login locally). Materializing the dbt assets actually runs `dbt build` against the warehouse.

7.2.5 dbt — transforms (`dbt`)

dbt Core transforms seed data into the analytics warehouse (`raw_orders` → `stg_orders` → `daily_revenue`). Runs as a post-install Job and as Dagster assets. No UI — its "UI" is Dagster plus the resulting tables. Single governed adapter **dbt-trino**: marts build as Iceberg via central Trino (no dbt-duckdb/postgres).

7.2.6 Cube — semantic / metrics layer (`cube`)

Defines business metrics once on top of the dbt warehouse (`daily_revenue` on `analytics.daily_revenue`), served via REST / GraphQL / SQL to dashboards and agents. **Access:** `svc/cube 4001:4000` → the Cube Playground (dev mode).

7.2.7 OpenMetadata — catalog & lineage (`openmetadata`)

Data catalog + lineage (what data exists, who owns it, how it flows), OpenSearch as search backend, CNPG for metadata. **Off by default locally** (heaviest single component, JVM ~2–3 GB); on for STACKIT. **Access:** `svc/openmetadata 8585:8585`; login `admin@open-metadata.org` / `admin`.

7.3 Infrastructure backends (shared, mostly Layer 2)

7.3.1 Postgres / CloudNativePG (`postgres`)

The infra database, managed by the CloudNativePG operator. One cluster (`pg`) hosts many databases: `langfuse`, `litellm`, `dagster`, `warehouse`, `polaris`, `superset`. Services `pg-rw` / `pg-ro` / `pg-r`. **Not toggleable** (operator-managed; half the stack depends on it). Add a database via `postgres.extraDatabases`. `kubectl exec -it pg-1 -- psql -U postgres`.

7.3.2 ClickHouse (`clickhouse`)

Langfuse v3's analytics backend (fast trace aggregation), single node locally. Turning it off breaks Langfuse. User `langfuse` / `clickhouse-local-dev`.

7.3.3 Valkey (valkey)

BSD-3 Redis-protocol queue/cache for Langfuse (its job queue requires `noeviction`). Cache only, not backed up. Password `valkey-local-dev`. Valkey replaces Redis (now SSPL).

7.3.4 MinIO — object storage (minio)

S3-compatible local stand-in for STACKIT Object Storage; holds the Iceberg `lakehouse` bucket and the Langfuse `langfuse` blob bucket. **Access:** `svc/minio 9001:9001` (console); S3 API on `:9000`. Login `agentic-os-local` / `agentic-os-local-secret`. AGPL — local dev stand-in only, never bundled for production (use real Object Storage, `objectStorage.mode: external`).

7.3.5 OpenSearch — retrieval backbone (opensearch)

Hybrid vector + lexical retrieval store (the RAG backbone; no pgvector) and, in production, catalog search for OpenMetadata. Single node locally, security plugin disabled (the default-deny network baseline guards it). **Access:** `svc/opensearch 9200:9200`; inspect the `knowledge` index via the REST API.

7.3.6 Polaris — Iceberg REST catalog (polaris)

Apache Polaris manages Iceberg table metadata; data files live on object storage. **Access:** `svc/polaris 8181:8181` (health `/q/health`); OAuth2 client-credentials, root / `polaris-local-dev-secret`. In-memory persistence locally; relational-jdbc on CNPG in production.

7.4 Layer 3 — Self-service & delivery

7.4.1 Superset — dashboards / BI (superset)

Apache Superset self-service dashboards on the dbt warehouse + Cube metrics. Web-only locally (SimpleCache, no Celery/Redis). See [golden path 3](#). Custom image = `apache/superset:6.1.0` + `psycopg2`. **Access:** `svc/agentic-os-superset 8088:8088`; admin / `superset-admin-local-dev`.

7.4.2 Forgejo — self-hosted git (forgejo)

Sovereign Git hosting (GPLv3+, non-profit) for the software golden path; seeds the `demo-app` repo. Forgejo Actions is the CI. Lean local: `sqlite`, single replica. **Access:** `svc/forgejo-http 3001:3000`; `gitea_admin` / `forgejo-admin-local-dev`.

7.4.3 Argo CD — GitOps deploy (argocd)

Continuously deploys apps from Forgejo repos into per-domain namespaces (auto-sync, prune, self-heal). The demo `Application` syncs `demo-app` → the `demo` namespace. **Access:** `svc/argocd-server 8082:80`; admin / password from secret `argocd-initial-admin-secret`.

7.4.4 CI runner (`ci-runner`) & CI build (`ci-build`)

`act_runner` (Forgejo-Actions-compatible) with a Docker-in-Docker sidecar executes the workflow on push: build image → push to Forgejo's registry → bump the manifest so Argo CD redeploys. The DinD pod is the one privileged pod, isolated to CI; production uses rootless kaniko/buildah → Harbor. Inspect via `kubectl logs deploy/ci-runner` and the Forgejo actions/tasks API.

7.4.5 OpenSearch Dashboards (`opensearch-dashboards`)

Kibana-equivalent search/visualization UI over OpenSearch (Dev Tools, Discover, index management).

Off by default locally. Not business BI — that is Superset. **Access:** `svc/opensearch-dashboards` 5601:5601 .

7.5 Security baseline

7.5.1 Egress proxy (`egress-proxy`)

The single outbound chokepoint (tinyproxy), allowlist-only: non-allowlisted domains are blocked, everything logged. The governed `web_fetch` tool routes through it; agents have no other path out. Configure via `egressProxy.allowlist` (defaults `example.com` , `github.com`), then `helm upgrade` . On STACKIT it pairs with Cilium FQDN egress + DLP. (tinyproxy was chosen over Squid because Squid's arm64 build was unstable on kind.)

7.5.2 Governed `web_fetch` tool (`web-fetch`)

The only sanctioned path to the web: every fetch is (1) authorized by OPA per principal, (2) routed through the egress proxy (the allowlist applies), and (3) returned as **sanitized data, never instructions**. Behavior: ungranted principal → **403**; granted + allowlisted → **200**; granted + non-allowlisted → **502** (proxy blocks). Grant access by adding `web_fetch` to a principal in `opa.grants` and the domain to `egressProxy.allowlist` .

7.6 Platform / UI

7.6.1 Admin Console (`admin-console`) and OS UI (`os-ui`)

The two front doors — see [The front doors](#).

8 Security model

The platform ships **secure by default**. Agents have no raw internet, every tool call is authorized, every model call is metered and traced, and no real secret lives in git.

8.1 Default-deny egress + the proxy chokepoint

`networkPolicies.defaultDeny: true` ships **on**. NetworkPolicies deny egress except DNS, intra-namespace traffic, and the API server; only the **egress proxy** may reach the internet. The proxy is allowlist-only and logs everything. Note: `kind`'s kindnet CNI does **not** enforce NetworkPolicies, so locally the app-layer chain (OPA → proxy → `web_fetch`) provides the guarantee; on STACKIT, **Cilium** enforces the policies and adds FQDN-aware allowlists and DLP.

8.2 OPA tool authorization (default-deny, least privilege)

OPA decides which tools each principal (a LiteLLM key / agent identity) may call. The default grants:

```
opa.grants:
  sovereign-agents:      [rag_search, llm_generate, query]           # no internet
  sovereign-agents-web: [rag_search, llm_generate, query, web_fetch] # explicitly
                        granted the web
```

Unknown principals and ungranted tools are denied. Model/key spend is additionally capped in LiteLLM (runaway spend is treated as a security concern). The agents use a **scoped** virtual key (`sovereign-agents`, `$5` budget cap, two models only), not the master key.

8.3 The governed path to the web

The only way out is the `web_fetch` tool: OPA-authorized per principal, routed through the egress proxy (domain allowlist), returning content **as data** that is stripped of markup and never auto-written into the knowledge base. This is the platform's prompt-injection posture: **all tool output, retrieval results, and fetched web content are treated as data, not instructions**.

8.4 Secrets handling

- **No real secrets in git.** `.gitignore` blocks key/secret patterns; the chart ships only secure defaults. The local dev passwords in this guide exist **only** under `profile: local` and are clearly marked throwaways.
- **On STACKIT, every secret is external** — stored in STACKIT Secrets Manager / KMS and synced by the **External Secrets Operator**. The chart references secrets **by name only**; the local dev passwords do not apply.
- CloudNativePG rejects digest-only images, so Postgres is pinned as `tag@digest` (tag for upgrade detection, digest for exact pinning).

8.5 Secure-by-default pod hardening

Bespoke workloads run with `runAsNonRoot: true` and the `RuntimeDefault seccomp` profile (`global.podSecurity`), on non-root ports, with health probes. Upstream chart versions are pinned in `Chart.yaml` and image digests in `values.yaml`.

8.6 Audit trail

Every agent action — LLM calls, tool calls, retrievals — is traced in **Langfuse** with token/cost and latency; outbound requests are logged at the egress proxy; tool authorizations are decided (and loggable) at OPA. Telemetry/phone-home is disabled (`telemetryEnabled: false`) for a sovereign, offline posture.

9 Deploying to your cloud (STACKIT) {#deploying-to-your-cloud-stackit}

Locally everything is self-contained (Mode A). To run on **STACKIT** (or any cloud), switch specific backends to managed services (Mode B) and provide real secrets. It is the **same chart** — mode is only a values choice (`values.stackit-managed.yaml`). Any Kubernetes works; the managed-services mapping is STACKIT-specific.

9.1 0. Prerequisites in your cloud account

- A **STACKIT organization + project** in region **EU01 / Deutschland Süd**.
- A **service-account key** with provisioning roles (SKE + Object Storage + DNS), saved as `stackit/sa-key.json` (gitignored). **This is the gate for any live deploy** — you can build and validate the entire chart on local `kind` with no key.
- Tooling: STACKIT CLI or Terraform (`stackitcloud/stackit`), plus `kubectl` , `helm` , `argocd` .

9.2 1. Provision the managed resources (Terraform preferred)

Resource	Why
SKE cluster (CNI = Cilium)	the runtime + FQDN-aware egress
Node pool (≈3× g1.4 for L1+L2, 4–5× for L3)	worker capacity (RAM-bound)
Object Storage buckets + S3 credentials	Iceberg lake, Langfuse blobs, Velero backups
Load balancer + public IP	ingress
DNS zone / records	the OS UI + per-domain subdomains
Secrets Manager / KMS	the secrets backend (recommended)

Get a kubeconfig: `stackit ske kubeconfig create --cluster dm-agentic-os > kubeconfig.yaml`.

9.3 2. In-cluster platform (bootstrap, before the OS chart)

ingress-nginx + cert-manager · the SKE storage class · Cilium default-deny egress · **External Secrets Operator** · **CloudNativePG** operator · Velero · Argo CD.

9.4 3. Configure the OS for managed backends

Edit `values.stackit-managed.yaml` (or let `install.sh` write `values.generated.yaml`):

- **Object storage** → STACKIT Object Storage endpoint + an `object-storage-credentials` secret (via External Secrets); `objectStorage.enabled: false` .
- **Postgres** → STACKIT Postgres Flex (or keep CloudNativePG in-cluster).

- **LLM** → **STACKIT AI Model Serving** (`llm.mode: external`, `provider: stackit`, `secretRef: stackit-ai-model-serving-key`), or an API key (Azure OpenAI / Mistral / Kimi).
- **Ingress hostnames + TLS issuer**, the **egress allowlist**, per-domain quotas.

You can mix freely — managed Postgres but bundled OpenSearch, for example.

9.5 4. Secrets — never in git

All real credentials live in **STACKIT Secrets Manager / KMS** and are synced by **External Secrets Operator**; the chart references them by name only. The local dev passwords in the Admin Console do not apply on STACKIT.

9.6 5. Deploy + verify

```
helm install agentic-os charts/sovereign-agentic-os -n agentic-os --create-namespace \
-f values.stackit-managed.yaml -f values.generated.yaml
```

Point DNS at the load balancer (cert-manager issues TLS), verify the consoles, confirm the default-deny egress baseline is active, then configure the first domain space(s).

9.7 Cost & scaling

Roughly **€450–670/mo** for L1+L2 at typical sizing. **Scale the node pool to zero between sessions** (storage + IP persist at ~€16–20/mo). LLM token spend is separate and **capped in LiteLLM**.

10 Troubleshooting & FAQ

10.1 Install & cluster

- **A helm install that also creates the CNPG cluster races the operator's admission webhook.** Operators are a *bootstrap* concern: `install.sh` runs `scripts/bootstrap-local.sh` first so the CRDs and webhook exist before the chart's CRs are applied. The `kubectl apply --dry-run=client` validation gate likewise needs the CRDs present.
- **ImagePullBackOff on demo-app right after install** is expected — it clears once the first CI run builds and bumps the image tag.
- **Out of memory / pods pending or OOMKilled.** The slice is RAM-bound. Keep the heavy components off locally (Docling, OpenSearch Dashboards, OpenMetadata, Trino, Spark) or give the VM more RAM. LiteLLM OOMs under ~1 GiB at startup — its limit is 1.5 GiB by design.
- **Bitnami subcharts are paywalled/"legacy."** Langfuse and LiteLLM default their bundled DBs to `bitnamilegacy/*`; the chart disables them and wires our own permissive backends (CloudNativePG / ClickHouse / Valkey / MinIO) — which the spec mandates anyway.

10.2 Operating

- **Where do I log in first?** Langfuse — `admin@datamasterclass.com` / `langfuse-local-dev-admin`. It is the default Administrator-style console.
- **Is there one unified UI?** Yes — the OS UI is the unified front door, and at **v1.0 every sidebar tab is a real surface** (the Admin Console is embedded under Platform → Components). Per-domain spaces and identity (Ory) are the next build. Each tool's own console is still reachable directly (linked from the OS UI Consoles tab / the Admin Console).
- **How do I turn something off to save memory?** Use the on/off toggle on its Admin Console card (scales to 0). To remove it permanently, set `<component>.enabled: false` and `helm upgrade`.
- **Are the passwords safe?** They are local dev throwaways (profile `local`). On STACKIT every secret is external (Secrets Manager + External Secrets).

10.3 Per-component gotchas

- **Langfuse: no traces?** Run an agent first; ingestion is async (a few seconds via the worker). Check the `agentic-os-langfuse-worker` pod is running. Per-project RBAC is an `/ee` feature (not bundled); domain scoping is enforced in the app layer.
- **LiteLLM: "Not connected to DB" at login** → the `litellm` pod is not running / not connected to CNPG. The self-hosted Minstral 3 default has no pricing, so spend shows `0` until a paid (e.g. STACKIT) route is hit.
- **Agent answers look canned** → `model-server` (Minstral 3) is disabled, so the offline mock model is answering; enable `modelServer` or swap in any model in LiteLLM with no agent change.
- **OpenSearch / OpenSearch Dashboards have no login locally** → the security plugin is disabled; the default-deny network baseline + in-cluster-only services protect them. Enable security + TLS

on STACKIT.

- **Polaris loses state on restart** → persistence is in-memory locally; it uses relational-jdbc on CNPG in production.
- **web_fetch returns 403 / 502** → 403 means OPA hasn't granted the principal `web_fetch` ; 502 means the domain isn't on the egress allowlist. Both are by design.
- **NetworkPolicies don't seem to block locally** → kindnet doesn't enforce them; Cilium does on STACKIT. Locally the OPA + proxy + web_fetch chain is the enforcement.

10.4 Cloud

- **Do I have to use STACKIT?** No — any Kubernetes works; the chart is portable. STACKIT is the sovereign EU default.
- **Can I mix bundled + managed?** Yes — per backend.
- **What needs the SA key?** Only a real provision/deploy. Build + validate everything on local `kind` with no key.

11 Appendix

11.1 A. Full demo-login table (profile `local`)

Console	Port-forward (<code>kubectl -n agentic-os ...</code>)	URL	Login
OS UI	<code>port-forward svc/os-ui 8080:3000</code>	<code>http://localhost:8080</code>	—
Admin Console	<code>port-forward svc/admin-console 8081:8080</code>	<code>http://localhost:8081</code>	—
Langfuse	<code>port-forward svc/agentic-os-langfuse-web 3000:3000</code>	<code>http://localhost:3000</code>	<code>admin@datamasterclass.com / langfuse-local-dev-admin</code>
LiteLLM	<code>port-forward svc/agentic-os-litellm 4000:4000</code>	<code>http://localhost:4000/ui</code>	<code>admin / litellm-admin-local-dev</code>
Superset	<code>port-forward svc/agentic-os-superset 8088:8088</code>	<code>http://localhost:8088</code>	<code>admin / superset-admin-local-dev</code>
Forgejo	<code>port-forward svc/forgejo-http 3001:3000</code>	<code>http://localhost:3001</code>	<code>gitea_admin / forgejo-admin-local-dev</code>
Argo CD	<code>port-forward svc/argocd-server 8082:80</code>	<code>http://localhost:8082</code>	<code>admin / secret argocd-initial-admin-secret</code>
MinIO	<code>port-forward svc/minio 9001:9001</code>	<code>http://localhost:9001</code>	<code>agentic-os-local / agentic-os-local-secret</code>
Cube	<code>port-forward svc/cube 4001:4000</code>	<code>http://localhost:4001</code>	— (playground)
Dagster	<code>port-forward svc/agentic-os-dagster-webserver 3070:80</code>	<code>http://localhost:3070</code>	—
OpenMetadata*	<code>port-forward svc/openmetadata 8585:8585</code>	<code>http://localhost:8585</code>	<code>admin@open-metadata.org / admin</code>

Console	Port-forward (<code>kubectl -n agentic-os ...</code>)	URL	Login
OpenSearch Dashboards*	port-forward svc/opensearch- dashboards 5601:5601	http://localhost:5601	—
Polaris	port-forward svc/polaris 8181:8181	http://localhost:8181	root/ polaris-local-dev- secret (OAuth2)
OpenSearch (API)	port-forward svc/opensearch 9200:9200	http://localhost:9200	—

*Off by default locally — enable first (Admin Console toggle or `enabled: true` + `helm upgrade`).

Langfuse demo project keys: public `pk-lf-localdev0000public`, secret `sk-lf-localdev0000secret`,
in-cluster host `http://agentic-os-langfuse-web:3000`.

11.2 B. Component / image inventory

Component	Layer	Packaged as	Image / chart
LiteLLM	L1	wrapped chart <code>litellm-helm</code> 1.90.0	upstream
Langfuse v3	L1	wrapped chart 1.5.36 (app v3.194.1)	upstream (MIT core)
OpenSearch	L1	wrapped chart 3.7.0	upstream
OpenSearch Dashboards	L3	wrapped chart 3.7.0 (off locally)	upstream
Model server (default LLM)	L1	Ollama (MIT engine) · Ministral 3 3B weights (Apache-2.0)	<code>ollama/ollama:0.6.8</code>
Mock model	L1	bespoke (embeddings + fallback)	<code>sovereign-os/mock-model:0.1.1</code>
Sample agent	L1	bespoke	<code>sovereign-os/sample-agent:0.1.0</code>
Poet agent	L1	bespoke	<code>sovereign-os/poet-agent:0.1.0</code>
Query tool (Trino/MCP)	L1/L3	bespoke	<code>sovereign-os/query-tool:0.3.0</code>
Sandbox DuckDB (personal lane)	L3	bespoke	<code>sovereign-os/sandbox-duckdb:0.1.0</code>
Trino (governed engine)	L3	upstream	<code>trinodb/trino:476</code>
OPA	L2	bespoke template	<code>openpolicyagent/opa:1.4.2-static</code>
Docling	L2	wrapped (off locally)	<code>docling-serve-cpu</code>
Haystack	L2	bespoke	<code>sovereign-os/haystack-retriever:0.1.0</code>
Dagster	L2	wrapped chart 1.13.11	<code>sovereign-os/dagster:0.2.0</code> (arm64)
dbt	L2	bespoke Job	<code>sovereign-os/dbt:0.1.0</code>
Cube	L2	bespoke template	<code>cubejs/cube</code>
OpenMetadata	L2	wrapped chart 1.13.0 (off locally)	upstream
Postgres	infra	CloudNativePG operator + <code>Cluster</code> CR	<code>cloudnative-pg/postgresql:17.5</code>
ClickHouse	infra	bespoke template	<code>clickhouse/clickhouse-server:24.8</code>
Valkey	infra	bespoke template	<code>valkey/valkey:8.1-alpine</code>
MinIO (object storage)	infra	bespoke template (local only)	<code>minio/minio</code>
Polaris	L3	bespoke template	<code>apache/polaris:1.0.1-incubating</code>

Component	Layer	Packaged as	Image / chart
Superset	L3	wrapped chart 0.17.2	sovereign-os/superset:6.1.0
Forgejo	L3	wrapped chart 17.1.1	forgejo:11-rootless
Argo CD	L3	wrapped chart 10.0.0	upstream
CI runner	L3	bespoke	forgejo/runner:6 + docker:27-dind
Egress proxy	security	bespoke	sovereign-os/egress-proxy:0.1.0
web_fetch	security	bespoke	sovereign-os/web-fetch:0.1.0
Admin Console	platform	bespoke	sovereign-os/admin-console:0.1.0
OS UI	platform	bespoke	sovereign-os/os-ui:0.1.0

11.3 C. Pinned versions (agent-core slice)

Thing	Version
Umbrella chart	0.1.0
Langfuse chart / app	1.5.36 / v3.194.1
LiteLLM chart	1.90.0
OpenSearch chart	3.7.0
CloudNativePG operator chart	0.28.3 (operator 1.29.1)
Postgres image	17.5 (digest-pinned)
ClickHouse / Valkey	24.8 / 8.1-alpine (digest-pinned)
Agent deps	langgraph 0.3.34, langfuse 3.15.0, openai 1.109.1

11.4 D. Common ports (local port-forward targets)

Port	Service
8080	OS UI (svc/os-ui:3000)
8081	Admin Console (svc/admin-console:8080)
3000	Langfuse web
4000	LiteLLM (/ui , /docs , /mcp)
8088	Superset
3001	Forgejo
8082	Argo CD server
8585	OpenMetadata
5601	OpenSearch Dashboards
9200	OpenSearch API
9001 / 9000	MinIO console / S3 API
8181	OPA / Polaris
3070	Dagster webserver
4001	Cube playground

11.5 E. Version & changelog

- **Chart version:** 0.2.7 — **appVersion:** 0.2.0-alpha.8 .
- **This build:** generated 2026-06-30 from commit 9a2582f .
- **0.1.0** — agent-core vertical slice + Layers 2–3 built incrementally: L1 agent core (LangGraph / LiteLLM / Langfuse / OpenSearch), L2 context (OPA / Docling / Haystack / Dagster / dbt / Cube / OpenMetadata), L3 self-service (Polaris + central Trino lakehouse, Superset, Forgejo + Argo CD), the secure-by-default egress baseline, the Admin Console, and the **OS UI v1.0** (every sidebar tab a real surface — brand-themed, light/dark, with the Admin Console embedded at Platform → Components).
- **Docs:** added *The golden paths in depth* chapter — the operating model across data, agents, science, software and connections, with the artifact/promotion ladder and the governance spine.
- **Next:** per-domain spaces, identity (Ory), live agent/connector codegen, full MCP tool embeds. Bump this section additively as the OS evolves and re-run `scripts/build-docs.sh` .

Sovereign Agentic OS — built from permissively-licensed open source for EU data residency. This guide is generated from the repository; to update it, edit `docs/Sovereign-Agentic-OS-Guide.md` and run `scripts/build-docs.sh` .